

CHAPITRE 4 : LES FONCTIONS

1. Présentation des fonctions:

Pour permettre de réutiliser une portion de programme sans réécrire les lignes, on utilise les fonctions. Lorsqu'on appelle une fonction, une portion de code est exécutée.

Définition :

Une fonction est un sous-programme nommé destiné à faire un traitement bien précis. Elle

- reçoit (en entrée) des informations : les arguments : données.
- renvoie (en sortie) une information, la valeur de retour.

La syntaxe de définition d'une fonction a la forme suivante :

```
type nomFonction( liste des types et noms des arguments)  
  
{  
  
    Corps de la fonction  
  
}
```

Où :

- **type** est le type de la valeur retournée,
- **nomFonction** est le nom de la fonction,
- la liste des types et nom des arguments séparés par des virgules,
- le corps de la fonction : les instructions entre accolades.

Exemple 1 : Fonction qui retourne une valeur

```
float moyenne( float x1, float x2, float x3)  
{  
    float moy ;  
    moy = (x1+x2+x3)/3;  
    return moy ;  
}
```

Description:

`moyenne` est le nom de la fonction

`float` est le type de la valeur retournée

`x1`, `x2` et `x3` sont des arguments (paramètres) de la fonction

`moy` est la valeur retournée à l'aide du mot clé `return` qui interrompe l'exécution de la fonction.

Appel d'une fonction :

```
#include<iostream>
using namespace std ;
float moyenne( float x1, float x2, float x3)
{
    float moy ;
    moy = (x1+x2+x3)/3;
    return moy ;
}
int main()
{
    float nMath=12, nPhys=11.5, nInfo=17.5;
    float note;
    note= moyenne(nMath, nPhys, nInfo) ;// Appel de la fonction
    cout<< note <<endl;
    return 0;
}
```

L'appel de la fonction `moyenne` est réalisé par l'instruction

```
note= moyenne(nMath, nPhys, nInfo) ;
```

où les arguments `x1`, `x2` et `x3` prennent les valeurs des variables `nMath`, `nPhys`, `nInfo` respectivement. La valeur retournée par la fonction `moyenne` est affectée à la variable `note`.

Exemple 2 : Fonction qui ne retourne pas de valeur

Une fonction qui ne retourne pas valeur est définie par **void** comme type de retour.

(void == RIEN).

Dans ce cas, on n'utilise plus le mot clé **return** dans le corps de la fonction.

```

#include<iostream>
using namespace std ;

void estPair( int n)

{

    if (n%2 == 0)
        cout<<n <<"  est pair"  <<endl;

}

int main()

{

    for(int i=0 ; i<100 ;i++)
        estPair(i);

    return 0;

}

```

La fonction principale main() appelle la fonction estPair(...) pour afficher les entiers pairs compris entre 0 et 99.

Exemple 3 : Tableaux comme arguments d'une fonction

```

#include<iostream>
using namespace std ;
const int NMAX=100;

void saisie( float T[NMAX], int n)//saisie de n éléments de T

{

    for(int i=0; i<n; i++)
    {
        cout <<"Entrez l element " <<i << ": ";
        cin >> T[i] ;
    }

}

```

```

float maximum( float T[NMAX], int n)//max de n éléments de T
{
    float  max =T[0];

    for(int i=1; i<n; i++)
    {
        if (max <  T[i])

            max = T[i] ;
    }
    return max ;
}

int main()
{
    float T[NMAX], m;

    int nbr;

    cout <<"Entrez la nombre d elements: ";

    cin >>nbr ;

    saisie(T,nbr) ;

    m= maximum(T,nbr) ;

    cout << "Le max est: " << m <<endl;

    return 0;
}

```

2. Variable globale et locale :

Une variable globale est déclarée à l'extérieure de toutes les fonctions y compris la fonction main(). Elle est alors accessible dans tout le programme.

Une variable locale est déclarée à l'intérieure d'une fonction. Elle est alors accessible uniquement dans cette fonction.

Exemple :

```
#include<iostream>
using namespace std ;

const float PI= 3.14 ;

float r; // Variable globale

float perimetreCercle( )
{
    float p;//Variable locale

    p = 2*PI *r;

    return p;
}

float surfaceDisque( )
{
    float s; // Variable locale

    s = PI *r*r;

    return s;
}

int main()
{
    cout << "Entrez le rayon:" ;

    cin >> r;

    cout << "Perimetre:"  << perimetreCercle() <<endl;

    cout << "Surface :" << surfaceDisque() <<endl;

    return 0;
}
```